

Course Registration Portal (Academia) Report

Name: Saheem Showkat Reshi

Roll no.: IMT2023051

1. Problem Statement

The problem addressed by this project is the development of a multi-user, client-server application for a course registration system. The system manages information for three distinct user roles: Administrator, Faculty, and Students. Each role has specific functionalities:

- Administrator: Manage student and faculty accounts (add, view, activate, block, modify).
- Faculty: Manage the courses they offer (view, add, remove, update details) and change their password.
- Students: Manage their course enrollments (view all courses, enroll, drop, view enrolled courses) and change their password. The system handles concurrent access to data securely and provides a reliable communication channel between the client and the server.

2. Implementation Details

The system is implemented using a client-server architecture in C, with file-based data storage and role-based access control.

Server (server.c):

- Acts as the central hub, listening for incoming client connections on a defined port (`PORT = 35043`).
- Uses multi-threading (`pthread`) to handle multiple clients concurrently. Each client connection is managed in a separate thread.

- Implements a signal handler (`handle_sigint`) to gracefully shut down the server upon receiving a `SIGINT` signal (e.g., Ctrl+C).
- Uses file locking (`fcntl`) to protect shared data files (`students.txt` , `faculty.txt` , `courses.txt`) from race conditions during read/write operations. Locks are applied at the file level.
- Authenticates users based on their role (Admin, Faculty, Student) and credentials using the `authorise` and `adminCheck` functions.
- Directs authenticated users to their respective menu functions (`adminScreen` , `facultyScreen` , `studentScreen`).

Client (client.c):

- Connects to the server using TCP sockets with a hardcoded IP (`127.0.0.1`).
- Provides a command-line interface for user interaction.
- Sends user input to the server and displays responses in real-time.
- Implements a simple protocol where the server sends prompts ending with `:` or `>` , and the client responds with user input.
- Includes a signal handler (`handle_sigint`) for graceful disconnection on `SIGINT` .

User Role Modules (Integrated in server.c):

- **Admin:**
 - Manages student/faculty accounts (add/view, activate/block, modify passwords).
 - Uses functions like `adminAdd` , `adminView` , `adminChangeStatus` , and `adminSetPassword` .
 - Data is stored in `students.txt` and `faculty.txt` with fields separated by `|` .
- **Faculty:**
 - Manages courses (add/remove/update, view offerings).
 - Uses functions like `facultyAddCourse` , `facultyRemoveCourse` , `facultyUpdateCourse` , and `facultyViewOfferingCourses` .
 - Course data is stored in `courses.txt` with format: `code|name|seat_limit|enrolled` .

- **Student:**

- Enrolls/drops courses and views enrollments.
- Uses functions like `studentEnroll` , `studentUnenroll` , and `studentView` .
- Course lists are stored as comma-separated values in `students.txt` .

Utility Functions (`utils.c`, `utils.h`):

- Provides common functions for socket communication (`sendString` , `recvString`) and file locking (`fileLock` , `fileUnlock`).
- Implements line-by-line string reception to handle fragmented TCP streams.
- Supports thread-safe file operations with read/write locks.

Common Definitions (`common.h`):

- Defines constants like `MAX_LINE` (buffer size) and `PORT` .
- Declares the `File` struct for managing file data in memory (lines, buffers, locks).

Data Persistence:

- Data is stored in plaintext files (`students.txt` , `faculty.txt` , `courses.txt`) with `|` as a delimiter.
- File locking ensures atomicity during concurrent access.
- The `save` function writes back changes to disk, while `load_file` reads and parses files into memory.

Key Features:

- **Role-Based Access Control:** Strict separation between Admin, Faculty, and Student functionalities.
- **Thread Safety:** File locks prevent corruption during concurrent access.
- **Error Handling:** Graceful shutdown on signals, input validation, and file operation checks.
- **Modular Design:** Functions are grouped by role, with shared utilities in `utils.c` .

The system prioritizes simplicity and reliability, using file-based storage with locking for small-scale concurrency.

3. Source Code Explanation

1. server.c

Core Server Functions

- `void handle_sigint(int sig)`
 - **Purpose:** Gracefully shuts down the server on `SIGINT` (Ctrl+C) by closing the server socket.
- `static int load_file(const char *path, File *file_struct, int lock_mode)`
 - **Purpose:** Loads a file into memory with thread-safe locking. Returns `1` on failure.
- `static void save(File *f)`
 - **Purpose:** Writes modified file content back to disk (e.g., after updates to `students.txt`).
- `static void free_file(File *f)`
 - **Purpose:** Releases memory and unlocks a file after use.

Authentication & Role Handling

- `static int adminCheck(int s)`
 - **Purpose:** Validates admin credentials (hardcoded as `Admin:11112222`).
- `static int authorise(int sockfd, const char *filepath, char *username)`
 - **Purpose:** Authenticates students/faculty against `students.txt` or `faculty.txt`.

Role-Specific Menus

- `static void adminScreen(int s)`

- **Purpose:** Displays and processes admin menu options (e.g., add/view users).
- `static void facultyScreen(int s, const char *faculty_user)`
 - **Purpose:** Handles faculty menu (e.g., add/remove courses).
- `static void studentScreen(int s, const char *studentName)`
 - **Purpose:** Manages student menu (e.g., enroll/drop courses).

Thread Management

- `static void *clientFunction(void *param)`
 - **Purpose:** Thread entry point for each client connection; routes to role-specific screens.
-

2. client.c

Client-Side Functions

- `void handle_sigint(int sig)`
 - **Purpose:** Closes the client socket gracefully on `SIGINT`.
 - `static int is_prompt_line(const char *s)`
 - **Purpose:** Checks if a server message ends with `:` or `>` (indicating a prompt for input).
 - `int main()`
 - **Purpose:** Connects to the server, sends user input, and displays server responses.
-

3. utils.c

File & Socket Utilities

- `int fileLock(int fd, int lock_type)`
 - **Purpose:** Applies `F_RDLCK` (read) or `F_WRLCK` (write) locks to files.
- `int fileUnlock(int fd)`

- **Purpose:** Releases file locks.
- `ssize_t sendString(int sockfd, const char *buf)`
 - **Purpose:** Sends a string over a socket.
- `ssize_t recvString(int sockfd, char *buf, size_t maxlen)`
 - **Purpose:** Reads a string from a socket until `\n` or EOF.

Helper Functions

- `static int lineSplit(const char *src, char *fld[4])`
 - **Purpose:** Splits a line (e.g., `chirayu|chirayu|1|CS101`) into fields using `|`.
 - `static int getRecvChoice(int sock)`
 - **Purpose:** Reads and parses an integer choice from the client socket.
 - `static int canSkipLine(const char *s)`
 - **Purpose:** Checks if a line is empty or a comment (starts with `#`).
 - `static int find_row(File *f, const char *key, int key_field)`
 - **Purpose:** Searches for a key (e.g., username) in a file's specified field.
-

4. Role-Specific Functions (server.c)

Admin Operations

- `static void adminAdd(int sockfd, const char *filepath, const char *role_label)`
 - **Purpose:** Adds a student/faculty record to `filepath` (e.g., `students.txt`).
- `static void adminView(int sock, const char *filepath, const char *header)`
 - **Purpose:** Lists all active users (students/faculty) from `filepath`.
- `static void adminChangeStatus(int sockfd, int activate)`
 - **Purpose:** Activates (`1`) or blocks (`0`) a student account.
- `static void adminSetPassword(int sockfd, const char *filepath)`
 - **Purpose:** Updates a user's password in `filepath`.

Faculty Operations

- `static void facultyUpdateCourse(int sockfd, const char *faculty_id)`
 - **Purpose:** Updates course details (name/seat limit) in `courses.txt`.
- `static void facultyAddCourse(int s, const char *prof)`
 - **Purpose:** Adds a new course to `courses.txt` and the faculty's course list.
- `static void facultyRemoveCourse(int s, const char *who)`
 - **Purpose:** Removes a course if no students are enrolled.
- `static void facultyViewOfferingCourses(int sockfd, const char *faculty_user)`
 - **Purpose:** Lists courses taught by a faculty member.
- `static void facultyChangePwd(int s, const char *who)`
 - **Purpose:** Updates a faculty member's password.

Student Operations

- `static void studentEnroll(int s, const char *user)`
 - **Purpose:** Enrolls a student in a course (updates `courses.txt` and `students.txt`).
- `static void studentUnenroll(int s, const char *user)`
 - **Purpose:** Drops a course and updates enrollment counts.
- `static void studentView(int s, const char *user)`
 - **Purpose:** Displays a student's enrolled courses.
- `static void studentChangePassword(int s, const char *user)`
 - **Purpose:** Updates a student's password.

Key Notes

- **File Handling:**
 - All file operations (read/write) are protected by `fileLock` / `fileUnlock`.
 - Data is stored in plaintext (e.g., `students.txt`) with `|` delimiters.
- **Thread Safety:**

- Each client connection runs in a separate thread (`pthread`).
- Global variables (e.g., `server_fd`) are minimized to avoid race conditions.
- **Error Handling:**
 - Functions return `1` or `NULL` on errors (e.g., file access failures).
 - Socket disconnections are detected via `recvString` returning `<= 0` .

4. Output Screenshots from Admin Menu

Starting server and client

```
saheem@pop-os:~/Desktop/iiitb/Imt2023
sem4/OS lab/mini-proj/IMT2023004_Min sem4/OS lab/mini-proj/IMT2023004_MiniPr
iProject/academia-portal$ ./server
Server listening on port 35043
█
```

```
see what I include pthread src/client.c src/defs.c & client
saheem@pop-os:~/Desktop/iiitb/Imt2023 sem4/OS lab/mini-proj/IMT2023004_MiniPr
oject/academia-portal$ ./client
.....Welcome Back to Academia :: Course Registration.....
..
Login Type
Enter Your Choice { 1.Admin , 2.Professor , 3.Student } :
█
```

Log in as admin


```

..
Login Type
Enter Your Choice { 1.Admin , 2.Professor , 3.Student }:
1
Enter username:
Admin
Enter password:
11112222
Admin authentication successful

..... Welcome to Admin Menu .....
1. Add Student
2. View Student List
3. Add Faculty
4. View Faculty List
5. Activate Student
6. Block Student
7. Modify Student Details
8. Modify Faculty Details
9. Logout and Exit
Enter Your Choice:

```

Adding a student

```

..... Welcome to Admin Menu .....
1. Add Student
2. View Student List
3. Add Faculty
4. View Faculty List
5. Activate Student
6. Block Student
7. Modify Student Details
8. Modify Faculty Details
9. Logout and Exit
Enter Your Choice:
1
Enter Student username:
saheem
Password:
sah
[OK] Added

```

Viewing student list

```

..... Welcome to Admin Menu .....
1. Add Student
2. View Student List
3. Add Faculty
4. View Faculty List
5. Activate Student
6. Block Student
7. Modify Student Details
8. Modify Faculty Details
9. Logout and Exit
Enter Your Choice:
2

Student List
- saheem      [active]
- Tom         [active]

```

Adding faculty

```
Enter Your Choice:
3
Enter Faculty username:
Henry
Password:
Ford
[OK] Added
```

Viewing faculty list

```
9. Logout and Exit
Enter Your Choice:
4

Faculty List
- Henry      [active]
- Rob        [active]
```

Changing student details

```
7. Modify Student Details
8. Modify Faculty Details
9. Logout and Exit
Enter Your Choice:
7
Enter Username:
saheem
New password:
ss
Password changed successfully
```

Output screenshots from Professor Menu

Faculty login and adding course

```

saheem@pop-os:~/Desktop/iiitb/Imt2023 sem4/OS lab/mini-proj/IMT2023004_MiniPr
oject/academia-portal$ ./client
.....Welcome Back to Academia :: Course Registration.....
..
Login Type
Enter Your Choice { 1.Admin , 2.Professor , 3.Student }:
2
Enter Username:
Rob
Enter Password:
Stark
Faculty authentication successful

..... Welcome to Faculty Menu .....
1. View Offering Courses
2. Add New Course
3. Remove Course from Catalog
4. Update Course Details
5. Change Password
6. Logout and Exit
Choice:
2
Course ID:
R1
Course Name:
Robots
Seat Limit:
10
Course added successfully

```

Viewing offered courses

```

..... Welcome to Faculty Menu .....
1. View Offering Courses
2. Add New Course
3. Remove Course from Catalog
4. Update Course Details
5. Change Password
6. Logout and Exit
Choice:
1
R1 (code) | Robots (name) | 10 (limit) | 0 (current enrollments)
R2 (code) | Math2 (name) | 80 (limit) | 0 (current enrollments)

```

Updating course details

```
Choice:
4
Enter Course Code to update:
R2
Enter new Course Name:
Stat Math
Enter new Seat Limit:
40
Course updated successfully.
```

Removing a course(can only remove if no students are enrolled currently)

```
Choice:
3
Course ID to remove:
R1
Course removed successfully
```

Output screenshots from student menu

Student login, enrolling in a course and viewing enrolled courses

```

object/academia-portal$ ./client
.....Welcome Back to Academia :: Course Registration.....
..
Login Type
Enter Your Choice { 1.Admin , 2.Professor , 3.Student }:
3
Enter Username:
saheem
Enter Password:
ss
Student authentication successful

..... Welcome to Student Menu .....
1. Enroll in Course
2. Drop Course
3. View Enrolled Courses
4. Change Password      (newPwd)
5. Logout and Exit
Enter Your Choice:
1
Course ID to enroll:
R2
Enrolled successfully

..... Welcome to Student Menu .....
1. Enroll in Course
2. Drop Course
3. View Enrolled Courses
4. Change Password      (newPwd)
5. Logout and Exit
Enter Your Choice:
3
Enrolled:

- R2 : Stat Math

```

Student dropping a course

```
..... Welcome to Student Menu .....
1. Enroll in Course
2. Drop Course
3. View Enrolled Courses
4. Change Password      (newPwd)
5. Logout and Exit
Enter Your Choice:
2
Course ID to drop:
R2
[OK] Unenrolled
```

Logging out

```
[OK] Unenrolled

..... Welcome to Student Menu .....
1. Enroll in Course
2. Drop Course
3. View Enrolled Courses
4. Change Password      (newPwd)
5. Logout and Exit
Enter Your Choice:
5
Logged out successfully
saheem@pop-os:~/Desktop/iiitb/Imt2023 sem4/OS lab/mini-proj/IMT2023004_MiniPr
```